

I assumed they were all the same, and I assumed Bill Gates's was the same as those.

So, back at work, I grabbed some HP BASIC manuals and studied them. I started writing on paper a syntax table. This is what describes the grammar of the computer language. It defines what commands a programmer can enter.

For instance, if English had a syntax table, it would explain that personal pronouns like "he" and "she" are nouns and usually subjects in a sentence such as, "He threw the ball." It would list all the possible verbs, of which "threw" would be one. And it would tell you what all possible "objects" would be, such as "ball." In English, there are millions of possibilities for subjects, verbs, and objects, but in a language like BASIC you can limit them to a certain number of items.

Then there are the rules you need. Say you wrote out the equation $5 + 3 \times 7$. When you write that out with no parentheses, a mathematician would know that you execute multiplication and division first, addition and subtraction second. So really that equation would be $5 + 21$. So that rule, about what terms to execute first, is an example of something that has to be defined in the syntax table.

I had no idea what other people did in their computer languages, but I felt it was obvious that you needed a noun stack to hold things like numbers, a verb stack (which would include actions like multiplication or addition), and a set of priorities for every single verb that was typed in.

It took me about four months to come up with the core of my BASIC interpreter. I ended up leaving out the ability to type in decimal numbers (called "floating point arithmetic"), and instead handled everything with integers—that is, whole numbers. That saved me about a month of work, I figured. I decided that for games and computer simulations—the two main things I was writing the BASIC for—I would just get by with integers.

Many of the key programs in my life, including those back in Colorado, used only integers. So I designed my BASIC to only work with numbers from $-32,768$ to $+32,767$.

I wrote the whole program on paper—with machine instructions on the left and the equivalent in hexadecimal (equivalent to 1s and 0s) on the right. I had to do this by hand because I couldn't afford to use an assembler program, which is the typical way you'd do this. This is the same way I had to write the little monitor program.

So I figured, Hey, I'm able to write the program with the code by myself, by hand. Who needs a computer to do this for you?

By the way, I still have the notebook I wrote my BASIC interpreter in. I'm not sure, but I bet it could be worth a lot to a museum now!

Anyway, the end result of all this was, when my 6502 BASIC was in the computer, I could type in little programs with the keyboard. Like I could have the computer ask you, "What's your name?" And if you typed it in, it would fly your name all around the screen. This sounds so simple now, but back then, nobody had ever seen a small computer where you could actually type in programs with a regular keyboard and have it execute.

Even with machines like the Altair, it was very expensive to add a plug-in card and a big cable that would connect to an enormous ugly teletype that had the keyboard to type on.

I showed my computer running BASIC at Homebrew after the main meetings a few times, and people were just blown away by it.

But there was one problem. The Apple I had no permanent storage—no hard disk drive or floppy or CD drive like you see today. This was way, way before that. So every time I wanted the BASIC to run, I had to turn on the computer and literally type my whole program in from my notebook. This was a 4,000-byte program—it took almost forty minutes for me to type it all in every time. And when I turned off the computer, because there was no

The Basics on BASIC

The BASIC computer language, the one I told you I sniveled at, was designed from the start to be an easy-to-program language for writing computer programs. Created in 1963 by Dartmouth College professors John Kemeny and Thomas Kurtz, the BASIC acronym stands for Beginner's All-purpose Symbolic Instruction Code. There's no question that BASIC is an easier-to-learn language than languages such as Pascal or C. It's also smaller and slower. But it worked just perfectly with my first Apple computers.

permanent storage and only RAM, the whole program would be gone. I ended up having to either leave my computer on all the time—which meant I couldn't transport it very well—or come up with a solution.

That's how I developed the cassette tape interface for the Apple I. Aside from changing the type of RAM from static to dynamic, this was the only change to my original design from the very first days of Homebrew in the spring of 1975. I designed a circuit so that a regular cassette tape could hold the BASIC, and when I turned the computer on, it would automatically load my BASIC into memory so the computer was usable.

Once the BASIC was done and easily loadable from a cassette tape, I discovered something terrible. I had miscalculated. I had thought that all versions of BASIC were more or less the same, and that all the 101 games in BASIC that I had in that book would automatically run if you typed them in. That turned out not to be the case. It turned out that the type of BASIC I'd written—as well as the HP BASIC I'd originally studied—was totally different from Bill Gates's Microsoft BASIC, which was based on the DEC BASIC at the time. Bummer!

So anyone who wanted to put those games on the Apple I was going to have to make changes to the games to do it.

I did manage to get some games working on the Apple I, though. There was a popular game, in BASIC, called Star Trek. Like the show. I adapted it to my BASIC and it ran just fine.

• o •

After we started selling the boards to Paul Terrell—working day and night to get them to him on time—we had profits like I never imagined. Suddenly our little business was making more than I was making at HP. That wasn't very much, admittedly. But still, it was a lot. We were building the boxes for \$220 and selling them wholesale to Paul Terrell for \$500.

And, of course, we didn't need a ton of money to operate. I had a day job, so I looked at it as, Hey, cool. Extra money for pizza! As for Steve, he was living at home. I was twenty-five and he was only twenty-one at the time, so what expenses could we have, really? Apple didn't have to make that much to sustain itself and be ongoing. We weren't paying ourselves salaries or paying rent, after all. We didn't have any patents to pay for. Or lawyers. It was a small-time business, and we weren't worried that much about anything.

My dad, watching this, pointed out that we weren't actually making money because we weren't paying ourselves anything. But we didn't care, we were having too much fun.

• o •

Right after we delivered to Paul Terrell, Steve arranged for me to show the Apple I PC board during the main meeting of Homebrew in about March 1976. I had shown off my computer after the meeting for months by then, but I'd never talked about it formally to the whole group.

Of course, I'd never spoken in front of a group of people this large. This was the largest spotlight I had ever been in. I mean, by now Homebrew had grown to about five hundred or more peo-

ple. The meeting was being held at the auditorium at the Stanford Linear Accelerator Center (SLAC). So I just stuck to the facts after I walked down the aisle with my printed circuit board in my hands. This was the first of only two times I ever spoke in front of the Homebrew meeting. (The other was when I introduced the Apple II.)

I knew that many people in the club had seen me running my prototype. So I just stood there in front of everybody and described the chips on the board—what they were and all—and talked about the specifications and the architecture. I talked about how I built it. And I talked about the main thing, as far as I was concerned: having a human-typable keyboard instead of a stupid, cryptic front panel with a bunch of lights and switches.

I explained that I'd used dynamic RAM instead of static RAM and why. I pointed out that my board had 8K bytes RAM and compared that to the Altair motherboard, which had only 256 bytes. I talked about a little BASIC program—the one that moved your name around on-screen when you typed it in. I described the video circuitry, the connectors, the voltages needed, everything. And finally I got to tell everyone the price—\$666.66.

I'm not sure if we were a big hit or not. You'll have to ask someone there who saw me giving the talk. After all, by that time a lot of Homebrew members were either starting or working for little computer companies. So maybe they couldn't see that the Apple I was that special.

But I could. And Steve could. We were so proud.

We were participating in the biggest revolution that had ever happened, I thought. I was so happy to be a part of it. It didn't have to be a big business. I was just having fun.

Ron Wayne, the third partner, wasn't having as much fun, I guess. He was used to big companies and big salaries. We bought him out for \$800 after we delivered some of the first boards to Paul Terrell and well before we got our first outside investment.

Chapter 13

The Apple II

By early 1976 we had sold maybe 150 computers. Not just through the Byte Shop but through other little stores that were popping up all around the country. We would drive around California, just walk into a store, and ask if they wanted to carry the Apple I. We did sell a few that way.

But this was nothing. Because we were watching other companies that sprang up around Silicon Valley at this time. And one of them, Processor Technology it was called, was supposedly selling more than a thousand units a month of their SOL-20 computer. It was the hit of the hobby computer world. It was also the hobby computer that supported a keyboard, which is how they designed it after I showed the Apple I at a main meeting at Homebrew. The Apple I started that trend.

Lee Felsenstein, the guy who emceed the Homebrew meetings, had actually designed the SOL. And Gordon French worked there. So we heard things.

I thought the Processor Technology SOL computer wasn't that impressive. Steve and I were sure we could sell more than what they were selling. But by then we had a prototype of the next Apple, the Apple II, and it was ten times better than the Apple I.

With that computer, we knew we could easily sell as many

computers as Processor Technology if we just had the money to build them.

The Apple II, which I started working on almost as soon as the Apple I was complete, was a phenomenal improvement over what I'd done before. I knew I wanted to have a computer that did color, for instance. I had built the Apple I from the beginning with chips working at the frequencies you would need to generate color on an American television, and I had planned to add color. But though I'd designed the Apple I so I could add a color to it, I decided it would be better to design a fresh computer instead.

You see, the add-on to color wasn't just a matter of buying more chips. It was a matter of efficiency and elegance of design. I wanted to design color from the ground up, not just as an add-on to an existing computer. That way, the Apple II would be designed with color ability on those chips from the start.

Another Apple II improvement I thought of was to design the whole new computer around text and graphics, with all of it coming out of the system's own memory.

So rather than having a whole separate terminal to do the on-screen stuff and other memory to do the other computations, I decided to combine all the memory into one bank—one section of DRAM. A portion of the DRAM the microprocessor used could also be continually tapped for whatever needed to be displayed on the screen.

In doing this, I knew I would save some chips. In fact, the Apple II ended up with half as many chips as the Apple I in the end.

It was also quite a bit faster. Remember how I told you how the Apple I had to constantly keep the contents of its DRAM memory alive by refreshing them? Well, by now I had faster DRAM chips. And instead of the microprocessor being able to access (write from or read to) the RAM once every millionth of a second, these new chips could do it twice every microsecond.

In fact, it even worked out that the microprocessor could access the RAM in one-half of a microsecond (millionth of a second) while the circuitry that refreshed the RAM could access during the other half. That's why the new computer I designed, this Apple II, actually ran faster. It was also smaller and cheaper. And that is always the goal with me.

The Apple II had countless improvements over the Apple I. Some people consider the Apple II to be a second design built from the Apple I, but I want you to know that that is not so. Not so at all.

The Apple I was not a computer designed from the ground up. It was a quick extension of my ARPANET terminal to a microprocessor, with virtually no electronic innovations except for the DRAM.

The Apple II, on the other hand, was designed and engineered from the ground up. Also solely by me.

Looking back now, I could've done the Apple II first—color and all—but I chose to go with the design I could come up with most quickly.

It's true that both machines brought striking advances to the computer world. The Apple I made history by being the first personal computer that could work with a keyboard and a display. But the Apple II brought color, high-resolution graphics, sound, and the ability to attach game paddles. It was the first to boot up ready to use, with BASIC already built into the ROM.

Other computers eventually caught up, but it took years for them to match what I'd done. Eventually every one of them would have to offer that same list of features.

The Apple II was the first low-cost computer which, out of the box, you didn't have to be a geek to use.

• • •

But no one had seen the Apple II yet. I was still finalizing it, and we were still working in our houses at this point. I was working out of my apartment and Steve was working the phones in his

bedroom. We were still testing computers in his garage. I was still building calculators at HP, and I still thought this was just a hobby. I was still planning on working at HP forever.

But it was very soon after delivering the Apple I boards to Terrell that I had a working Apple II. And like I said, it wasn't just twice as good. It was like ten times better.

By August 1976, I had completed it—the board, I mean, which was the center of the Apple II. I remember that so well because that was the month Steve and I flew out to the PC '76 show in Atlantic City.

• o •

We got on the plane in San Jose, and Steve and I sat together with the Apple I and II with us on board. And the funny thing was, a bunch of the people we knew from Homebrew, who now worked at all these little competing computer companies, were seated around us on the same plane. We could hear them talking in advanced business talk—you know, talking about proposals and using businesslike acronyms we'd never heard before. We felt so left out of these discussions.

But inside, we knew we had a secret. A big secret. Maybe we weren't part of the business-type groups, but we knew we had a better computer. Actually, we had two better computers. The Apple I and the Apple II. And no one in the world knew about the Apple II yet.

When the show started in Atlantic City, I was lucky because I didn't have to hustle the Apple I at the booth. I'm not a sales type. Steve Jobs and Dan Kottke did that. I was upstairs getting the very last BASIC sequences finished up.

The show was full of young, barely financed companies like Apple. The proprietors looked like us. I mean, there weren't any nicely dressed company executives, company owners, or company managers really attending the show. It was a pretty sloppy group of people, come to think of it.

They were in our business and most of them were competitors. We were all friends, but we were still competitors.

Even though we didn't let the Apple II out of the bag at that show, there was one guy not associated with any of these companies or businesses who saw it. He was a convention guy setting up a projection TV for the convention goers. Steve and I went down the first night, after everyone else had left, and met with this projector technician. I think we had told him to stay. It was probably about 9 p.m. You see, I had this different method of generating color and I was still amazed at how many TVs it worked with. In fact, I never found a TV that it didn't work with. But I figured that a projector might have different color circuitry that would choke on my color method. I wanted to see if the Apple II would work with it.

So I hooked the Apple II prototype up to this projector and it worked perfectly. That technician, who was seeing every low-cost computer in the world as he was setting up the show, told me that of all of them, this was the only computer he would buy.

I only smiled. The Apple II wasn't even announced yet.

• • •

After the show, the biggest, earthshaking Eureka moment ever was the day I got Breakout, the Atari game, working on the Apple II.

I had put enough capability in BASIC that you could read where the game paddles were. It could sound the speaker as needed, and it could plot colors on the screen. So I was ready.

I sat down one day with this little blank board with chips on the top side of it and little red and blue wire-wrapped wires all soldered underneath and connected it with some wires to transformers and then connected it all to my color TV.

I sat down and started typing in BASIC the commands I needed to make one row of bricks—just like the ones in Atari's arcade game—and it worked! I had a row of bricks. I played

around with different color combinations until I had the brick color that worked.

I made eight rows of bricks lying side by side. I figured out the right colors, I figured how the bricks should be offset to look more realistic. Even and odd rows. And then I started programming the paddle. I made the on-screen paddle go up and down with the game control knob. And then I put in a ball. I started giving the ball motion. Then I started telling the ball when it hits bricks, here's how it gets rid of the bricks and here's how it bounces. And when it hits the paddle, here's how it bounces and here's how it changes direction vertically and horizontally.

And then I played with all these parameters and it only took a half hour total. I tried dozens and dozens of different variations of things until, finally, I had the game of Breakout completely working on the Apple II, showing the score and everything.

I called Steve Jobs over. I couldn't believe I'd been able to do it, it was amazing. I sat him down and showed him how the game came up with the paddle and the bricks. And then I said, "Watch this." And I typed a couple of BASIC statements, changed the color of the paddle, and the color of the bricks, and where the score was.

I said, "If I had done all these varieties of options in hardware the way it was always done, it would've taken me ten years to do. Now that games are in software, the whole world is going to change."

That was the exact moment it sank in. Software games were going to be incredibly advanced compared to hardware games—games that were hardwired into arcades and systems like that.

These days, the graphics are so great in games. They have gotten incredibly complicated and huge in size. If they had to be in hardware, there wouldn't be enough time in the universe to design them.

I thought, Wow. Nobody in the club is ever going to believe

that an arcade game could be written in BASIC. It was a first in the world. I put a secret into my Breakout game for the Apple II, too. If you hit CTRL and Z on the keyboard, the game went into a mode where the paddle would always jiggle but could never miss the ball.

What a great feature. It tricked people into thinking they were just really lucky in hitting it. The paddle was so shaky and jiggly that a person could never tell it wasn't really because of their own skill and their own movements that they were hitting it.

One day I sat down with John Draper (Captain Crunch, remember?). We were at Homebrew right after the main meeting, the time people could demo stuff.

John had never played an arcade game before.

I said, "Here. Play this game." I showed him how you turned the dial so the paddle went up and down. And he sat there and played it. Everyone in the room watched him for about fifteen minutes. The ball was going so fast, and he, even though he didn't really know what he was doing with the control, kept hitting it. People just thought he was a superior game player.

After about fifteen minutes, he finally won the game. And all of us were congratulating him like he was the best game player in the world. I don't think he ever knew it was a setup.



In the spring of 1976, as I was working on the Apple II, Steve and I got into our first argument. He didn't think the Apple II should have eight slots. Slots are connectors you can plug extra circuit boards into in case you want to expand the functionality of the computer. Steve wanted only two slots—one for a printer and one for a modem. He thought that way you could build a cheaper, smaller machine that was good enough for today's tasks.

But I wanted more slots, eight of them.

I had the idea that there would be a lot of things people would want in the future, and no way did we want to limit people.

Usually I'm really easy to get along with, but this time I told him, "If that's what you want, go get yourself another computer." There wasn't a single chip I could save by reducing the number of slots from eight to two, and I knew people like me would eventually come up with things to add to any computer.

I was in a position to do that then. I wouldn't always be. A couple of years later, Apple went on to design the Apple III, which was just a disaster, and it had fewer slots.

But in 1976 I won that argument, and the Apple II was designed and eventually came out the way I wanted it to.

• • •

I remember coming in one day to HP—where I was still working—and showing the other engineers the Apple II. I demoed it doing color swirls. The other engineers would come up to me and say this was the best product they'd ever seen. And yet HP still couldn't find a way to do it right, a way to do this kind of project.

One day my boss, Pete Dickinson, told me that some people in my calculator division had created a new project that had gotten through levels of corporate approval to build a small desktop machine with a microprocessor, DRAM, a small video screen, and a keyboard. They even had five people assigned to write BASIC for it.

The awful thing about this was they all knew what I had done with the Apple I and even the Apple II. Yet they had started up this project without me! Why would they do that? I don't know. I think they just saw what they wanted to do as a project was what I'd done.

But I went to talk to the project manager, Kent Stockwell. Although I had done all these computer things with the Apple I and Apple II, I wanted to work on a computer at HP so bad I would have done anything. I would even be a measely printer interface engineer. Something tiny.

I told him, "My whole interest in life has been computers. Not calculators."

After a few days, I was turned down again.

I still believe HP made a huge mistake by not letting me go to its computer project. I was so loyal to HP. I wanted to work there for life. When you have an employee who says he's tired of calculators and is really productive in computers, you should put him where he's productive. Where he's happy. The only thing I can figure is there were managers and submanagers on this computer project who felt threatened. I had already done a whole computer. Maybe they bypassed me because I had done this single-handedly. I don't know what they were thinking.

But they should've said to themselves, "How do we get Steve Wozniak on board? Just make him a little printer interface engineer." I would've been so happy, but they didn't bother to put me where I would've been happiest.



Like I said before, we needed money. Steve knew it and I knew it.

So by that summer of 1976, we started talking to potential money people about Apple, showing them the Apple II working in color in Steve's garage.

One of the first people we showed it to was Chuck Peddle. Remember him? He was the guy from MOS Technologies who'd sold me the 6502 processor I designed the Apple I around the year before at the WESCON show.

By this time Chuck was working at Commodore, a consumer electronics company rumored to be shopping around for a personal computer to sell. I remember I was so impressed to meet him after the role his chip, the MOS 6502, had played in the Apple I. We'd opened Steve's garage to the sunlight that day, and he came walking in wearing a suit and a cowboy hat. Wow, I was excited to see him and couldn't wait to show him the Apple II. This was a very important person in my mind.

I typed in a few BASIC programs, showed some color spirals

on-screen, showed him how many chips and how it worked and everything. Just to show him what we were doing. Chuck was in good spirits throughout the meeting, laughing and smiling. He told us we should make a presentation to the company bigwigs, which we did a few weeks later.

I'll never forget how, in that conference room, Steve Jobs made what I thought was the most ridiculous statement. He said, "You might just want to buy this product for a few hundred thousand dollars."

I was almost embarrassed. I mean, there we were, we had no money, we had yet to prove to anybody there was going to be any money in this thing. Steve added, "A few hundred thousand dollars, plus you have to give us jobs working on this project."

Well, we left and heard back a few days later that, no, they'd decided they would build their own machine, it was cheaper. They didn't need to support fancy things like color, sound, and graphics, all the cool things we had. Chuck Peddle, in the garage, had told us he thought it was possible for them to do their own computer in four months. I didn't see how anyone could, but I guess after he saw the Apple II, it would be a lot easier to design something like what he wanted.

I saw the Commodore PET, the computer they came up with so quickly, a few months later at the West Coast Computer Faire, by the way. It kind of sickened me. They were trying to do something like what we'd shown Chuck in the garage that day, with a monitor and programming and a keyboard, but they made a real crappy product by doing it so quick. They could've had Apple, you know? They could have had it all if they'd had the right vision. Bad decision.

It's funny. I think back on it now—the Apple II would turn out to be one of the most successful products of all time. But we had no copyrights or patents at all back then. No secrets. We were just showing it to everybody.

• • •

After Commodore turned us down, we went over to Al Alcorn's house. He was one of the founders of Atari with Nolan Bushnell, and he was the one who'd hired Steve to do video games there two years before.

Now, I knew Al knew me. He knew I had designed Breakout, the one-player version of Pong. I remember that when we went to his house I was so impressed because he had one of the earliest color projection TVs. Man, in 1976, he would have been among the first people to have one. That was cool.

But he told us later that Atari was too busy with the video game market to do a computer project.

A few days after that, venture capitalists Steve had contacted started to come by. One of them was Don Valentine at Sequoia. He kind of pooh-poohed the way we talked about it.

He said, "What's the market?"

"About a million," I told him.

"How do you know?"

I told him the ham radio market had one million users, and this could be at least that big.

Well, he turned us down, but he did get us in touch with a guy named Mike Markkula. He was only thirty, he told us, but already retired from Intel. He was into gadgets, he told us. Maybe Mike would know what to do with us.

• • •

The very first time I met Mike, I thought he was the nicest person ever. I really did. He was this young guy. He had a beautiful house in the hills overlooking the lights of Cupertino, this gorgeous view, amazing wife, the whole package.

Better still, he actually liked what we had! He didn't talk like a guy who was hiding things and thinking about ripping you off. He was for real. That much was obvious right away.

What a major thing this was.

He was truly interested. He asked us who we were, what our backgrounds were, what our goals were with Apple, where we thought it might go. And he indicated some interest in financing us. He was talking about \$250,000 or thereabouts to build 1,000 machines.

Mike was just talking in normal commonsense terms about what the future of a new home computer industry might be like. Now, I had always thought of the Apple computer as being something for a hobbyist who wanted to solve a work simulation or play a game.

But Mike was talking about something different. He talked about introducing the computer to regular people in regular homes, doing at home things like keeping track of your favorite recipes or balancing your checkbook. This was what was coming, he said. He had a vision of the Apple II as a real home computer.

Now, we'd already been kicking around this idea a little, of course. I mean, out-of-the-box and ready-to-use was something Paul Terrell at the Byte Shop had asked for. And we were planning on doing that, as well as a plastic case. We had even planned to hire a friend of Steve's, Rod Holt, to build a switching power supply. That kind of power supply was so much more efficient than what was previously available—we knew it would generate less heat. That was necessary if you were going to fit a board and our power supply into a plastic case.

But when Mike agreed to sign up, he told us, "We're going to be a Fortune 500 company in two years. This is the start of an industry. It happens once a decade."

I believed him only because of his reputation and position in life, you know? He was the sort of person who if he said it—and you can tell sincerity in a person—he really believed it. I thought Fortune 500 might be out of the range, though. I mean, a \$5 million company would be immense and unbelievable.

But if somebody knows how to make certain judgments better

than I do, I don't try to use my own logic and reasoning to challenge them. I can be skeptical, but if someone really knows what they're talking about, they should be trusted.

It turned out that even Mike was underestimating our success. But look, I'm getting ahead of myself.

• • •

Well, after Mike agreed to do our business plan—after he started working on it—he asked to talk to me. He said, “Okay, Steve. You know you have to leave Hewlett-Packard.”

I said, “Why?” I mean, I'd been at HP the whole time I'd designed the Apple I and Apple II. And all the time I was moonlighting, I set up interfaces, did the color, the graphics, wrote the BASIC, just did the whole thing. I said, “Why can't I keep doing this on the side and just have HP as my secure job for life.”

But he said, “No, you have to leave HP.” He didn't give me any reasons. He told me I had to decide by Tuesday.

And I went and thought and thought and thought. I realized I had a lot of fun designing computers and showing them off at Homebrew. I had fun writing software and I had fun playing with the computer. I realized I could do all those things for the rest of my life. I didn't need my own company.

Plus, I felt very insecure in starting a company where I would be expected to push people around and run their affairs and control what they did. I'm not a management kind of person. I told you before: I'd decided long ago that I would never become someone authoritative.

So I decided I wouldn't do Apple after all. I would stay at HP for my full-time job and design computers for fun.

I went to the cabana—Mike had a cabana on his property—on ultimatum day and told Mike and Steve what I'd decided. I told them no. I'd thought about it, and I'd come to the conclusion that I wasn't going to leave HP.

I remember Mike was very cool about it. He just shrugged and

said, "Okay. Fine." He was really terse about it. It was like he thought, okay, fine, he would just get what Apple needed somewhere else.

But Steve was upset. He felt strongly that the Apple II was the computer they should go with.

• o •

Within a couple of days my phone started ringing. I started getting phone calls at work and home from my dad, my mom, my brother, and various friends. Just phone call after phone call. Every one of them told me I'd made the wrong decision. That I should go with Apple because, after all, \$250,000 is a lot of money.

It turned out that Steve had talked them all into calling me. Apparently he thought I needed an intervention.

But it didn't do any good; I still was going to stay at HP.

Then Allen Baum called.

Allen said, "Steve, you know, you really ought to go ahead and do it. Think about it. You can be an engineer and become a manager and get rich, or you can be an engineer and stay an engineer and get rich." He told me he thought it was absolutely possible for me to start a company and stay an engineer. He told me I could do it and never get into management.

That was exactly what I needed to hear. I needed to hear one person saying that I could stay at the bottom of the organization chart, as an engineer, and not have to be a manager. I called Steve Jobs right away with the news. He was thrilled.

And the next day I came in early, walked over to a couple of friends at HP, and told them, "That's it, I am going to leave HP and start Apple."

Then I realized, Oh, you should always tell your boss first. So I went over to tell him quickly, but he didn't show up at his table. I waited and waited, and finally it was like four in the afternoon, and he still wasn't at his table. Everybody kept coming up to me

as I waited there, saying, “Hey, I hear you’re leaving,” and I didn’t want my boss to hear it from someone else.

Finally my boss showed up near the end of the day. I told him I was leaving to start my own company. He asked me when I wanted to go. I told him, “Right away.” So he took me over to human resources and they interviewed me and all of a sudden I was gone. It was that quick.

But I never doubted my decision. I mean, I’d made my decision. Apple was the main thing for me from that point on.

• o •

Just before we met Mike, Steve and I made plans to move Apple from his house and my apartment to a real office. We had something like \$10,000 in the bank from the Apple I sales, so we were able to do this. The office was on Stevens Creek Boulevard in Cupertino, just a few blocks away from where the huge Apple campus would eventually be on Bandley Drive.

Then, when Mike joined us, we had even more money in our account. We moved into our little office. There were about five or six desks around. There was a little room to set up a lab bench to do some testing and debug work. It was a real long lab bench. And we had our key staff in place. Steve, me, Mike Markkula, Rod Holt, and, now, a guy named Mike Scott.

We’d hired Mike Scott to be the president just before Mike Markkula got there. (So now we had two Steves and two Mikes.) Mike, or “Scotty,” as we called him, was a guy with experience running things. He came from National Semiconductor, where he’d been a director.

I think a lot of people have forgotten him today, but Mike was Apple’s president and leader for four years—he took us public four years later.

We had this idea that we would announce and show the Apple II at the West Coast Computer Faire, which was about four months away. The Faire, started by Jim Warren, another

Homebrew member, would be in San Francisco in January of 1977.

So I had four months to finish things up. I was completing the 8K bytes of code we had to release to Synertek, the company that was going to make the ROMs for the Apple II. Those were the ROMs that would make it an Apple II running BASIC.

Then there was the project surrounding the plastic case. We were going to be the first computer ever in a plastic case. I did not have to deal with this at all, thank god. It was a rough project. Steve Jobs, Rod Holt, and Mike Scott dealt with that. They had a guy in Palo Alto who was signed up to make plastic cases for us. The process was time-consuming and laborious, and it turned out there was a limit to what this guy could do. He was using a process to build the cases, but it turned out he could only do a really small number per day.

It was only about three days before the West Coast Computer Faire that we got our first three plastic cases as samples. They came in, and we actually assembled the whole complete computer with the board inside. It sort of looked like what the Apple II would look like, and now we could show it off at the Faire.

Finally, in the days before the West Coast Computer Faire, Mike Markkula explained how we would all have to dress up nicely, how we should appear and look, how we should act. He coordinated how we would talk to people and show them things.

Of course, on the side, I started thinking of how I could do a prank at the show. First, I wrote a little joke program that would tell jokes about people's ethnicity. Then I set up a huge prank that would take a lot more effort than just a regular joke. And I thought I would play this joke on the big company that started it all for me. I'm talking about the company that made the Altair: the MITS Corporation.

Well, we had a list of everyone who was going to show computers and equipment at the West Coast Computer Faire, and I thought it was so strange that MITS wasn't going to be there.

I thought, What a great opportunity to pull a prank on them!

I got an idea from something I'd read in the Pentagon Papers. There was a part in there all about political trickery and a guy named Dick Tuck who played dirty tricks, clever little psychological tricks like putting out fake memoranda, fake notices to alarm people that were written in such a way that they couldn't really be denied as being lies. So I decided I'd put out a fake memo of my own—a fake ad, like a leaflet, for a fake product from MITS. After I heard from Mike Markkula that we were going to hand out twenty thousand brochures for the Apple II, I realized it would be possible to get thousands and thousands of fake ads out.

The first thing I did was call Adam Schoolski, who was a thirteen-year-old phone phreak when I'd first met him a few years earlier. He'd gone by the handle Johnny Bagel. Anyway, I told him that I wanted to do this prank but didn't want to do it near the San Francisco Bay Area. I had a lot of good experience with pranks by then, and I knew that you don't get caught if you keep people out of the loop, you don't do things nearby, and you keep a level of secrecy. And this was going to be a major prank, I told Adam, because I wanted to print up eight thousand leaflets to distribute. I was able to come up with the \$400 I needed to print eight thousand copies on different colors of paper.

Adam and I made it together. The product we made up was called the Zaltair. You see, there was a new company at the time, called Zilog, that was making a chip that was compatible with the Intel 8080. It was called the Z-80, and at the time there were lots of hobby computers coming out that were built around it. They were called the Z-this and the Z-that. All these companies were

always using Z words. So I came up with the Zaltair, a made-up computer that was also built around the Z-80.

I came up with all kinds of dumb computer-y Z words, too. Like Bazic. And Perzonality. Then I needed copy for the leaflet. I looked in a computer magazine, *Byte*, for an ad that was the worst ever. And I found it. It was from a company called Sphere. And it said dumb stuff like, “Imagine this. Imagine that. Imagine some other thing.” So I wrote copy that said, “Imagine a race car with five wheels.” I made up the stupidest things any idiot dork would laugh at, but if they saw it in a nicely done leaflet with good fonts, they would think it’s all real and legitimate. Imagine something going faster than the speed of light. Imagine a banjo with six strings. I came up with the dumbest things.

Also, I made a play on what was called the S-100 bus, the connection the Altair used to plug in expansion boards. I named the Zaltair’s equivalent the Z-150 bus. I wrote, “We have 150 slots. We call it the Z-150 bus.” I even said it was compatible with the S-100 bus, but with 50 extra pins. If you think about this, these are just the dumbest statements, but I knew people would read stuff into it as if these were fantastic advances, just because our leaflet was going to look so professionally created.

Then I decided to make this prank on the MITS Corporation look like Processor Technology had done it. After all, they did a competing computer, the SOL. I got this idea from my experience at the University of Colorado, when I was able to make it look like another guy had jammed the TVs in my class. Two pranks for the price of one! So the way I did this was I made up a totally phony quote that would raise anybody’s eyebrows. They would think, Whoa. What the hell is he saying? I attributed it to the president of MITS, Ed Roberts, and put it at the top in italics.

The quotation was completely and utterly nonsensical: “Predictable refinement of computer equipment should suggest online

reliability. The elite computer hobbyist needs one logical option-less guarantee, yet.” You see? The first letter of each word in those two sentences spells out Processor Technology!

Then, on the reverse side of the paper, I put a comparison chart. That was the common way magazines like *Byte* would compare computers. How fast are they? How big are they? How much RAM do they have? What processor do they use? Well, in my chart I made up the dumbest categories. Like, I had a category just generally called “hardware.” A computer would get from 1 to 10. Then, software. I rated these computers on uniqueness, on personality, on just the dumbest, most generic terms you would never see a computer rated on. I gave the Zaltair a 1 in every category, of course, and I always made the Altair come in second. Then all the computers that were actually better than the Altair would rate behind that. That way, it looked like they were worthless by comparison, even though anybody at that show probably knew that the competitors were so much better. Of course, I included the Apple II.

I hoped it would look like MITS was lying in their comparison chart anyway.

I realized that, man, this was so big, and there was no way I could get caught at this thing. I couldn't let it happen. I had two young friends, Chris Espinoza and Randy Wiggington, who knew about it—they were teenagers back then. And I told each of them that, no matter what, they could never tell anyone about it. Even if you get called by the police and they tell you that your partner told them everything, you should still deny everything. We are going to stonewall this, I told them, and never ever admit it to anyone.

Adam Schoolski lived in Los Angeles but he came up for the Faire. And when the four of us got there with the eight thousand handouts, we saw these huge tables where all the companies were putting out their brochures and flyers. We brought two thousand in at first and just set them on a table like what we were

doing was normal. And then we went around the Faire, kind of chuckling a little.

But Adam came up to me an hour later and told me the whole box was gone. Carton and everything. Gone.

So we went to our hotel and got another box of two thousand and brought them in. We stood around and watched until eventually some guy walks up, looks at one of the handouts, then picks up the box and takes it away. So we realized that a representative was there from MITS after all, intercepting them!

Now, we went back to the hotel and got a bunch more handouts. This time we didn't just put the carton down. Instead we carried them in our hands, under our coats, in our backpacks, and we put them in pay phones, corners, tables and everywhere. All over the Faire. We would find stacks of handouts—other companies' real handouts—and slip a few of ours underneath. So if somebody ran over right away, they wouldn't think we had slipped in bad stuff. Onesy, twosy, and we didn't get caught.

Thank god Steve and Mike didn't find out I'd done this. Mike, at least, would've said, "No, don't do pranks. Don't do jokes. They give the wrong image to the company." That's what any professional type would've said. But hey, they were dealing with Steve Wozniak. I did take work seriously—I had engineered a fantastic product, and everyone knew it—and I was serious about starting a company and introducing a product. But to me, that goes along with having fun and playing jokes. I'd spent my whole life like that. If you think about it, even a lot of the personality of the Apple computer was about fun. And that really came about just because my style was to have fun. Jokes make things worth doing.

I couldn't stop laughing the next day at Apple when Steve saw the comparison chart and started talking all positively about how we actually didn't perform too badly in the comparison. We were pretty lousy, of course, like everyone else but my made-up Zaltair, but he said, "Hey, we didn't do too bad, because, after all, we rank

better than some of the others.” Oh my god. Randy Wiggington had to run out of there, we were in such tears of laughter!

And the next night, which was the regular Wednesday night Homebrew meeting, I couldn’t wait to see if people had caught on. Sure enough, someone held my thing up in the air and started talking about the Zaltair, saying he’d called the company and this was a fake. A hoax.

It turned out about a third of the people in there, a couple hundred, had actually gotten the handout. So it did get around.

About a week later, Gordon French, who started Homebrew and by now had left his job at Processor Technology, was kicking around Apple to see if there was any consulting work he could do for us. I remember thinking he was just such a nice, pleasant, easygoing guy.

I took the opportunity and said to him, “Oh, did you ever hear about that Zaltair that got introduced?” I could barely hold my laughter when I asked him.

“Oh yeah,” he said. “That hoax. And I know who did it.”

Randy and I both perked up at this. I said, “Who? Who did it?”

He said, “It was Gary Ingram at Processor Technology. He has a strange sense of humor.”

This was exactly what I’d hoped for! Someone else getting the blame—and that someone else happened to be at our rival, Processor Technology. So it was a success.

I said, “You know, I heard there was kind of a code in the handout.” And I pulled the brochure out and looked at the letters like I was discovering this for the first time. “P . . . R . . . O . . . C . . .”

I’m sure that for years and years after this, everyone thought Processor Technology had done it. I never admitted it to anyone until many years later, when I was at a birthday party for Steve Jobs.

It was there that I presented him with a framed copy of it. As soon as he saw it, Steve broke up laughing. He’d never even suspected I’d done it!

The Biggest IPO Since Ford

Right after we officially incorporated as Apple Computer Corporation in early 1977, Mike had us go down to Beverly Hills and talk to patent lawyers. They said that any ROM chips we had around that had any code in them—any PROMs, any EPROMs—every single one of them needed a copyright notice. I had to put “Copyright 1977” on them all.

I sat down with one of the patent lawyers, Ed Taylor, and went through all the clever things in my design that other people definitely wouldn’t have done before. How I did the color, for instance, and how I did the timing for the DRAM.

We ended up with five separate parts of a patent. It was a good, secure patent that was going to wind up being one of those patents in history that become very, very valuable. It was going to be the heart of lawsuits to come. For instance, it would come in handy when people tried to copy, or clone, the Apple II and other products after that.

Back then, there were no ideas of how software could be patented. This was such new stuff. We found out that copyrights were a better way to deal with people copying our technology. Copyrights were an easier, quicker, and less costly way than patents to stop people who tried to copy our computer outright.

• • •

Soon after the West Coast Computer Faire, where we introduced the Apple II, a couple of other ready-to-use personal computers came out. One was the Radio Shack TRS-80, and the other was Commodore's PET. These would become our direct competitors.

But it was the Apple II that ended up kicking off the whole personal computer revolution. It had lots of firsts. Color was the big one.

I designed the Apple II so it would work with the color TV you already owned. And it had game control paddles you could attach to it, and sound built in. That made it the first computer people wanted to design arcade-style games for, the first computer with sound and paddles ready to go. The Apple II even had a high-resolution mode where a game programmer could draw special little shapes really quickly. You could program every single pixel on the screen—whether it was on or off or what color it was—and that was something you could never do before with a low-cost computer.

At first that mode didn't mean a lot, but eventually it was a huge step toward the kinds of computer gaming you see today, where everything is high-res. Where the graphics can be truly realistic.

The fact that it worked with your home TV made the total cost a lot lower than any competitors could do. It came with a real keyboard to type on—a normal keyboard—and that was a big deal. And the instant you turned it on, it was running BASIC in ROM.

As I said, Commodore and Radio Shack within a few months came out with computers that also ran BASIC out of the box. But the Apple II was far superior to them. The Radio Shack TRS-80 and Commodore PET did have DRAM like the Apple II, but they were limited to only 4K bytes of it. The Apple II could expand up to 48K bytes on the motherboard, and even more in the slots.

The TRS-80 and PET only came in 4K or 8K models, and they weren't expandable. The Apple II had eight slots for expansion; the other two had none. Finally, the PET and TRS-80 screens were black-and-white. No color like ours. And they had rickety keyboards with small keys.

The Apple II could grow into the future and had so much versatility built in. That's why it became the leader.

• • •

The Apple II was also an ideal computer for anybody who wanted to design a computer game.

We provided documentation and tools, making it really easy for programmers to create games in BASIC (at a hundred commands a second), or machine language (at a million commands a second), or both. The only way you could create a game for computers like the PET and TRS-80 was strictly in BASIC, and only with text characters on-screen. Unlike the Apple II, these machines didn't have graphics. It was inconceivable that anyone could've created a compelling arcade game on any of those computers.

Within months dozens of companies started up and they were putting games on cassette tape for the Apple II. These were all start-up companies, but thanks to our design and documentation, we made it easy to develop stuff that worked on our platform. Generally these little companies amounted to little more than a single guy in his house who figured out how to write a neat game, wrote it, and copied it onto a bunch of cassette tapes that he then sold through specialty computer stores.

And back then, there wasn't software rip-off happening like there is today. The stores weren't, for example, taking one cassette tape and making a bunch of copies and not paying the original guy. None of that was going on because there wasn't much money in the business yet. Ethics could still be high. It wasn't as if there was that much more money to be made by stealing.

So all the tapes the stores sold were legitimate, and the stores

were taking a cut on the games they sold. Within a year a whole Apple II industry sprang up with dozens and dozens of companies of little guys—just onesy-twosy companies, really—at home writing software for the Apple II.

And then little companies started building circuit boards that fit into the Apple II slots. Boards were easy to design for the Apple II because we gave complete documentation on how our boards worked. Also, I had included some great tools: the Apple II had a little operating system developers could access, as well as a set of easy-to-use software debugging tools I had written myself.

So how do you design a printer board that will attach a printer to the Apple II? How do you design a scanner or a plotter board? It was all so well documented that within a year of the Apple II shipping that June, all of a sudden there were all these Apple II add-on products being sold by people.

People who wanted to do an add-on board not only had to design that board, but they had to write a little program—a device driver program that translates between computer programs and the actual hardware. The predecoded addresses I had for all eight slots would be connected to a ROM or PROM chip on the board containing this program. The program could be 256 bytes long with just a single PROM chip, but each slot had another 2K bytes of predecoded address space for a larger amount of code. You had to be aware that this second address space went to every board, so in order to use it, there had to be some other circuits that knew which board was in control.

Otherwise, when one of these 2K bytes of addresses came along, a bunch of boards would put data to the processor, and the boards would conflict. Each board also had 16 predecoded addresses intended to trigger hardware—to control and sense the hardware devices.

There were so many options available to a board designer that

it led to a lot of very creative designs. The best designs made the most of the least, just as I like to do.

The computer magazines had tons of Apple II product ads for software and hardware. Suddenly the Apple II name was everywhere. We didn't have to buy an advertisement or do anything ourselves to get the name out. Our name was suddenly all over the place. We were just out there, thanks to this industry of software programs and hardware devices that sprang up around the Apple II.

We became the hot fad of the age, and all the magazines (even in the mainstream press) started writing great things about us. Everywhere you looked. I mean, we couldn't buy that kind of publicity. We didn't have to.

• • •

Like I said, the Apple II used a cassette tape for data storage. I had never been around or even used a floppy disk in my life. They did exist, though. I'd heard of floppy disks you could buy for Altair-style kit computers, and, of course, the expensive mini-computers of the time used them. Now, all these were in the big, eight-inch floppy format. That means they were spinning magnetic disks that measured eight inches in diameter. And you could only hold about 100K bytes of data on each floppy disk. That's not very much by today's standards. Totally, it's only about 100,000 typed characters.

But Mike Markkula told me at a meeting that we really should have a floppy disk on the Apple II. He was annoyed at the way it took forever to get his little checkbook program to load from cassette. A floppy disk, because it spins so much faster and stores data more densely, would load the checkbook program much more quickly.

For instance, a computer could read 1,000 bits per second off a tape, but it would go 100,000 bits per second off a floppy.

I knew that the Consumer Electronics Show (CES) in Las

Vegas was coming up. It would be the first CES where companies could demonstrate computers, and only marketing people from Apple were going.

I asked Mike, if I finished the disk drive in time, could I go to Vegas for the show, too? He said yes.

That gave me only two weeks to build a floppy drive for the Apple II, a device I had never seen working before or ever used in any way, but I now had this artificial motivation (artificial, because of course I could've gone to the CES if I wanted to) to try to astound people at Apple again.

I worked all day, all night, through Christmas and New Year's trying to get it done. Randy Wiggington, who was actually attending Homestead High by now, the school Steve and I had graduated from, helped me a lot on that project.

• • •

To help me get started, Steve told me he'd heard that a company named Shugart, which was the main floppy drive manufacturer at the time, was coming up with a 5-inch format. (Alan Shugart had invented the floppy years before when he was at IBM.) Steve was always looking for new technologies that had an advantage and were likely to be the trend, and this was definitely a case like that.

He got one of the new Shugart 5-inch drives for me so I could see if I could make it work with the Apple II. What I had to do was design a controller board—a card that would plug into the Apple II—that would let you read and write data from the floppy. The first thing I did was examine the drive and its controller board and how they worked. I scanned the manual. Finally I studied the schematics of their circuitry, and I analyzed Shugart's floppy disk circuit too. It had a connector and a protocol for how signals would be applied to write data. In the end, I decided that of the twenty-two or so chips, about twenty of them weren't needed. To make the floppy disk work required a combination of a circuit I

had to design and the existing circuit on the Shugart drive. I stripped out twenty of their chips, so that reduced twenty of my total end product. That's the way I always think about things. I could run data right from my own floppy controller to the read/write head and implement any start/stop protocols of my own in code on the computer. To tell you the truth, it was less work on the computer than generating the funny protocol Shugart wanted. Then I sat down and came up with a very simple circuit that would write data at floppy disk rates and read it. This turned out to be a real challenge.

• • •

In the case of the cassette tape interface I'd designed, I had a signal that constantly varied from high to low and low to high and so on. The signal could never stop as long as the tape was running. The circuit handling signals to a cassette recorder weren't designed to let a signal stop changing.

And the tape wasn't able to store a signal that stayed the same for too long. So I had the microprocessor time the low-high-low transitions according to the 1s and 0s of the data being written. I chose the rates of this cassette data to be between 1,000 and 2,000 hertz. Those were typical voice frequencies that a cassette tape was designed to record and play. That's approximately one millisecond (a thousandth of a second) between transitions from high to low to high and so on signals.

But the signals to a floppy disk needed transition times that were much shorter—only four to eight microseconds (or millionths of a second). There was no way to get my microprocessor to generate these timings directly from the 1s and 0s. It was just too fast. After all, the 6502 microprocessor inside the Apple II ran at a clock speed of approximately 1 MHz. The fastest instruction took two microseconds and would take many instructions to generate the timing for 1s and 0s. This was a problem.

I came up with an answer, thankfully.

The Apple II was designed to read and write bytes of data to cards plugged into the eight free slots, and it could do that really efficiently. So I came up with a scheme to output 8 bits (that's one byte) of data to the floppy controller, which would output those bits every four microseconds on its own, one bit at a time.

The 8-bit data code came from 4 bits of real computer data. I used a lookup table to do this efficiently.

Even so, it was barely possible for a perfect program, a program I had to write to myself, to keep up with this rate. And I had to count the exact number of clock cycles, in microseconds, for every step. That way, when I output 8 bits of code data to the controller, exactly every thirty-two microseconds, it matched the rate it had to be written onto the transfer rate. No matter what paths my program took, how many instructions, how many branches, how many loops, it always happened exactly every thirty-two seconds, when the next batch had to be written.

This sort of precision timing is a software job only a hardware person could deal with. Software programmers don't have to deal with precise timing ever.

• • •

This is as tricky as code gets. Even a minor change in the microprocessor could have killed it. For example, if they came out with a version of the 6502 that took three microseconds for a particular instruction instead of four, it would have screwed up all my calculated timing and the floppy controller for the drive would no longer work.

The floppy controller card had to accept 8 bits and merely shift it out to the floppy disk, through a magnetic write head, similar to writing to a cassette tape. That was how you saved data to the floppy, and that part was easy. An 8-bit shift register (registers hold data) could be loaded off the bus and do the four-microsecond shifting of the data.

Coming back from the other direction, reading data from the floppy was more of a challenge. I came up with the idea of creating a tiny processor—a tiny microprocessor, actually—I had to implement as what is called a state machine.

I did this using two chips, which was a remarkable achievement. One chip was a register and one was a PROM. I think I used a 6-bit register. Some of its bits were like maybe six of the 1s and 0s in this register, corresponding to a particular “state” the machine might be in. They functioned as address bits to a PROM.

The PROM would take as address inputs the bits indicating the current state—from the register—and also the bits of data from the floppy disk. Every microsecond, this PROM would output as data the next state number (which might be the same) and also a couple of bits that controlled the 8-bit shift register. That would shift the 0s and 1s to it in the appropriate time—when the time came to make decisions. That next state number would be reloaded into the register that held the state number.

Basically this little state machine was analyzing what was coming from the floppy disk every microsecond, and saving it in the master 8-bit chip register. Don’t confuse this 8-bit shift register with the register that held the state number for the state machine.

I had to fill the state machine PROM with 1s and 0s that caused the right actions in my machine. This was much more difficult than writing a program on a microprocessor, because every 1 and 0 had a specific, important meaning on that PROM.

I completed the state machine and was sure it would work. It was elegant—in fact the whole design was elegant, and I was proud of it.

Now, all this data (1s and 0s) was coming from the floppy disk, but I had to be able to determine which 0 or 1 would be the start of a byte. (Remember, 8 bits together form a byte.) Also, when this four- and eight-microsecond timing between transitions

came from the floppy disk to my controller, I didn't know which of the 0s and 1s was the start of a byte.

I was scared for a week or so as I built my controller that I wouldn't be able to solve this. But I did come up with some abnormal patterns that could be written onto the floppy disk but didn't translate back to data.

I would write about sixteen of these patterns in a row, and when they read back into my state machine, they automatically kept shifting it in time until it lined up with where the bytes would actually be. Then my read program, in the computer, continually looked for a couple of start bytes, called "mark bytes," which I'd write to indicate the start of a small section of data, called a "sector." Along with the data for each sector, I'd write the sector number of data on the floppy so that the reading program would be sure it was writing the correct sector. (If the reading sector ever determined the data was wrong, it would try again.)

• • •

For the floppy, I did the hardware design and the state machine coding. I also wrote the very tightly timed code to read and write specially coded data to and from the floppy disk. This was my forte.

Randy Wiggington wrote a higher-level routine, more useful to application programmers and operating system programmers.

After I was able to read and write data, I wrote routines to step the head to any of the thirty-six tracks on the floppy disk. It would step for a long time to position itself on track 0, the innermost track. Then I'd give a sequence of pulses to a stepper-motor to step the head to track 1, then track 2, and so on, to get to the point where the data it needed was. I had to wait a certain time between this track-stepping, as specified by Shugart.

At one point it occurred to me that moving the magnetic read/write head was like moving a heavy car. It has inertia. It's slow to get started, but once you get it moving it rolls on its own

inertia and you can push it to make it faster, and then faster yet. I decided I could probably safely accelerate the head as it crossed multiple tracks and then decelerate it to get it going slow enough not to overshoot the last track. Even if it overshoot, it would read the track number it got to and then back up.

I experimented and came up with a table of acceleration/deceleration numbers for timing that worked fine. Now, instead of a click-click-click sound like the burst of a machine gun, as the head moved around it made a nice-sounding whoosh. We had the fastest floppy disk access times in the industry because of this.

This sounds very complicated, I know. But it had very few parts. Making it work was incredibly hairy. It was one of those things you don't even know is possible. You can get an idea of just how much I sweated in those two weeks.

I know all this got really technical, but I had to explain because to this day, engineers are always walking up to me and saying how great that floppy controller I did was. And in just two weeks. Now you know how I did it.

• o •

The code was just about to the point where we could type "R Checkbook" to run the checkbook program or "R Color Math" to run the math program. I didn't actually have a floppy disk operating system in two weeks, but we had a table on the disk that tracked and sectorized each program occupied. Normally an operating system would read an index to the whole disk and when you requested "Color Math" it would look up in the index the list of tracks and sectors it occupied. We didn't have this quite working the day we had to fly to Las Vegas, but Randy and I were sure we'd have it done in a few hours after we got there.

So we got on the plane in San Jose and flew to Las Vegas.

• o •

That was a night Randy and I will never forget. It was our first time seeing the lights of Las Vegas. We were stunned by it. It was

a much different and smaller strip than today, with much smaller hotels. There weren't even as many of them as there are now, and many were old and tiny. But it was impressive. We'd never seen anything so lit up, that is for sure!

Our motel was the cheapest place in town, the Villa Roma. It was near Circus Circus and we learned the route from there to the Las Vegas Convention Center. Randy and I did a lot of walking that night. I showed Randy, who was seventeen at the time, how to play craps and he won something like \$35. At the convention center, we watched the late-night setup of all the booths. We set up in our booth and worked until about 6 a.m., finally getting everything working.

At that point I did one very smart thing. I was so tired and wanted some sleep but knew it was worth backing up our one good floppy disk, with all the right data.

I had some short programs that allowed me to read and write entire tracks. The floppy disk had thirty-six tracks. I decided to make a copy of this one floppy disk we had worked so long and hard to prepare. I only had two floppy disks with me so I decided to copy the good one to the blank. I inserted the good floppy and entered some data into memory to cause it to read track 0. Then I put in the blank floppy and used that data to write "track 0" on it. I did the same sequence for all thirty-six tracks. Backing up is smart, I always say.

But when I finished this backup, I looked at the two unlabeled floppy disks and got a sinking feeling that I'd followed a rote pattern but accidentally copied the bad floppy to the good one, erasing all the good data. A quick test determined that this is what happened. You do things like that when you are extremely tired. So my smart idea had led to a dumb and unfortunate result.

It meant that we would not, due to our tired state, have the floppy ready to show when the CES started in a few hours. What a bummer.

We went back to the Villa Roma motel and slept. At about 10 a.m. I woke up and got to work. I wanted to try to rebuild the whole thing. The code was all in my head, anyway. I managed to get the good program reestablished by noon and took it to our booth. There we attached the floppy and started showing it.

I can't tell you how successful and noted it was at this show, particularly in comparison to the Commodore PET and Radio Shack TRS-80, which were at the CES as well.

• o •

The floppy disk made the computer fast, but it was a program named VisiCalc that made it powerful.

Two guys in Boston, Bob Frankston and Dan Bricklin, worked closely with Mike Markkula to design it. And boy, was this the right product at the right time! And it was definitely the right program for the right machine.

VisiCalc was a software product for business forecasting—it was designed to answer “what-if” scenarios. For instance, if we sell \$100,000 worth of product X, how much revenue will we get? What if we sell half that? It was the earliest software program for doing spreadsheets on a personal computer, so that regular people working in business really had a high-tech tool.

And VisiCalc was so powerful it could only run on the Apple II. Only our computer had enough RAM to run it. The Radio Shack TRS-80 and the Commodore PET definitely weren't powerful enough. But we had the RAM, we had graphics on-screen and a two-dimensional display, and we were easy to use out of the box. And VisiCalc came out not on cassette, but on floppy disk. What a match.

Our business just exploded when VisiCalc came out. And the Apple II market suddenly moved from hobbyist people playing games who didn't mind waiting a few minutes for the program to load from tape, to business-type people who could load VisiCalc instantly.

After a couple of months, the businesspeople were something like 90 percent of the market. We had totally missed this audience, we never thought of it. But it took Apple in a whole new direction.

From 1,000 units a month, suddenly we went to 10,000 a month. Good god, it happened so fast. Through 1978 and 1979 we just got more and more successful.

By 1980 we were the first company to sell a million computers. We were the biggest initial public offering since Ford. And we made the most millionaires in a single day in history up to that point.

I believe the whole reason for this was the combination of the Apple II, VisiCalc, and the floppy disk.

• • •

Remember when I told you Mike had us copyright the software? Well, what a good move.

After the CES, we found out about a new computer from a company called Franklin. It supposedly looked a lot like ours. It arrived at our building, and it looked so much like the Apple II I was very interested.

I thought, Hey, great. They copied my design. I wonder how much of it they copied. I didn't expect they would've copied much of it. I figured engineers are trained to invent and design their own things. An engineer would never look at another person's design and copy it, would they? No, that's what they go to school for. They go to learn how to design their own things.

I walked over to the main building to look at it. There it was, and I was shocked. The printed circuit board inside was exactly the same size as ours. And every single trace and wire was the same as ours. It was like they'd taken our Apple II board and Xeroxed it. It was like they'd just Xeroxed a blank Apple II board and put in the exact same chips. This company had done something no honorable engineer would've done in their effort to make their own computer.

I couldn't believe it.

Well, at the next computer show I attended, I immediately went up to their booth and told the president, who was in the booth, "Hey, this is just a copy of ours." I was all upset.

"This is ridiculous," I told him. "You copied our board. You just copied it. Which means I am your chief engineer, and you don't even give me credit for being your chief engineer."

The president looked at me and said, "Okay. You're our chief engineer."

And I was happy and walked away, but now that I think of it, I should've asked him for a salary!

We did sue them later, and I found out their argument for doing it. They claimed there were legal reasons that gave them the right to copy the Apple II. They were arguing that because there was such a huge software base of programs for the Apple II, it was unfair to exclude them. They claimed they had a right to build a computer that could run that software base, but that argument sure didn't make sense to me.

The case took a couple of years. They lost, and we did get money from them. Just a few hundred thousand dollars, not the millions I thought it was worth. But it was enough to stop them.

More About the Floppy Disk

The floppy disk was invented by Alan Shugart in 1967, when he was working at IBM. The first floppies were 8 inches across—and they were called floppies because they were on a thin, bendable piece of magnetic material. Later, floppies went to a smaller format, the 5.25-inch format.

Later on, when they were in an even smaller 3.5-inch format and in a nonbendable plastic cover, people started calling floppies "diskettes."

The Woz Plan

Just before we went public in late 1980, a guy called me and asked whether he could buy some shares of my stock at \$5 a share. He wanted to buy 10 percent of it.

I loved the idea, because it meant I could afford to buy Alice and myself a house. We were still living in the Park Holiday Apartments in San Jose, paying a rent of \$150 a month.

But I like to do things different. I valued the employees at Apple—there were more than a hundred by then—as a community. I'd had that philosophy about a company being like a community since my first job, and maybe even earlier.

So I decided it would be better to sell some of my stock to employees and let them benefit, rather than some outside investor.

It was apparent to a lot of people at this point that Apple was going to have a very successful IPO—that, reasonably, the stock was going to be worth a whole lot more than \$5, at any rate. And top executives and founders at Apple had a lot of stock. We were all likely to make millions. But a lot of other employees were left out, the majority of them.

I decided I was going to offer to sell some stock really cheap to people who deserved it. Regular employees didn't get all the

stock options the executives got. Which wasn't fair. So I came up with something I called the Woz Plan. Any engineer or marketing person could buy 2,000 shares from me at a really low price of \$5.

Almost everybody who participated in the Woz Plan ended up being able to buy a house and become relatively comfortable. I'm glad of that. But at first our lawyers told me I wasn't going to be allowed to sell stock to all these people. They told me they had to be sophisticated investors or something. But finally our attorney, Al Eisenstat, said, "Okay, Steve, you can do it."

Then there was the matter of some of our earliest employees who didn't get stock at all. Randy Wiggington, who'd helped me do the floppy disk, had been there before we started Apple even. Chris Espinoza, Dan Kottke, and my old neighbor Bill Fernandez were other examples. These employees weren't just around, they offered the inspiration that really allowed me to do the great stuff. I thought of them as part of the family, part of the family that had helped me design the Apple I and Apple II computers.

I gave each of them stock worth about a million dollars.

In those days, giving stock away to people you thought deserved it was just unheard of. Companies at the time just didn't give stock to all the workers. They were like, "Why should we give these people stock? They did what they did for what they got paid and they didn't have stock." Never would any company go back and say, "Okay, well, you were real nice. So now I'll give you some stock." So this was different because I was giving them my own stock—like a gift—it wasn't coming from the company.

• • •

I think behind the scenes Steve thought I was weak because of this—sort of ditching the company a little bit in kind of a sellout. But I sold that stock at about \$5 a share to forty people in the Woz Plan—2,000 shares a pop—and then I was able to buy a really nice house for me and Alice. I bought it in cash. I figured, once you own a house, it's great. All you have to worry about is

the maintenance of the house if you don't have a job or anything. So I bought it and owned it outright.

It wasn't a very big house, but it was a nice house. It was probably the very favorite house of my life. It was just beautiful—located in the middle of the Santa Cruz Mountains, in Scotts Valley. It was an all-wood house—knotty pine with the holes in the wood. There was a big master bedroom upstairs. And I remember I could walk upstairs through the bedroom and walk out onto a balcony and look down into the family room and a little aviary with a bunch of windows. I had a gate out front with a wooden mural I had done of dogs. I got my first huskies there. I loved everything about that house.

Alice and I didn't stay together in that house very long, though. Even though we now had money like we'd never dreamed of, it wasn't enough to make up for the fact that we had different interests. She wanted to go out every night with her friends. I wasn't into that. I wanted to stay home and work. I didn't want to get divorced—I never wanted to get a divorce, ever. I'm the kind of guy who always wanted to get married to someone forever, and I wanted that with Alice.

But what could I do? I mean, by then the Apple stock was worth so many hundreds of millions of dollars, and she just told the counselor we were seeing that she wanted to see who she was without me and be on her own in life. She never once said to the counselor that I worked too hard, which is kind of a myth about my divorce that got in the press later. No, that's not what she said. She said she wanted to be on her own.

Let me tell you that I opposed that divorce as strongly as I could. I never wanted to get divorced in my life. But finally I realized there was no way to stop it. So I just took Alice to a park in Cupertino and wished her well and said goodbye. I walked back to Apple feeling really different. Different like it was time to move on. Alice was gone.

• • •

By this time Apple had its own building on Bandley Drive. By 1981 computers all of a sudden became the happening thing in life. There were articles in newspapers and magazines and TV shows talking about computers; it was just immense. Computers, personal computers, and home computers—suddenly everyone was wondering if they were going to make our lives better in the future, lead to better education systems, cause us to be more efficient and more productive. It looked like computers were going to enlighten us, improve our brains, let us think less and get correct answers sooner.

There was also a constant stream of articles in the trade press comparing our product with others on the market, and because we were the best technically, we were always rated as the best product. We were the one everyone wanted the most.

There were also stories about how we were just two people, Steve and I, and how we'd started with nothing and suddenly were so successful. We got all this publicity and all the benefits from it. Sales. Fame. We were just the hot, shining star.

In December of 1980, Apple's stock went public on the NASDAQ exchange.

• • •

It was the most successful IPO up to that time. It was on the front page of every major newspaper and magazine. Suddenly we were legendary. And rich. Really rich.

This was a pretty amazing accomplishment. After all, we had started from virtually nothing. It turned out Mike Markkula was right. We really were going to be a Fortune 500 company in five years.

Just a year later, we were going head to head with IBM's first personal computer, the IBM PC. Nonetheless, we had this major, major IPO. We also had the Apple III, a machine targeted to business, coming out—there were just rumors of it—and I think that

was part of the reason the timing was right. (Another reason was that because so many people had received shares, the reporting hassle to the SEC was harder than going public!)

That computer, the Apple III, was a strong, strong statement to the business world. It was like, after this incredible Apple II phenomenon, we were suddenly going to be able to compete with the then-new IBM PC.

• • •

The Apple III had some terrible problems, though. It was nothing like the Apple II, which was reliable all the time. I'm serious. You could buy an Apple II on eBay today, and it'll work. There is no modern product that is as reliable. In every speech I give, I talk to people who are still running Apple IIs, and they say those machines are still running after this many years.

No, the Apple III had hardware problems, serious ones. It would get to a store, for instance, boot up a couple of times, and then it would crash. Sometimes it wouldn't even boot up at all. My brother had a computer store by this time in Sunnyvale, and he told me Apple engineers would come down to fix it, but they could never get a machine that worked. Never. The first few months of the Apple III went by, and many of the stores had the same experience. Every Apple III came back not working. And what do you do when you're a computer dealer and this happens? Well, you stop selling it and you keep selling the original machine, the Apple II. That's why the Apple II was going to be the largest-selling computer in the world for at least three more years. In fact, by 1983 it would hit a huge milestone—it was the first computer to sell a million units!

So why did the Apple III have so many problems, despite the fact that all of our other products had worked so great? I can answer that. It's because the Apple III was not developed by a single engineer or a couple of engineers working together. It was developed by committee, by the marketing department. These

were executives in the company who could take a lot of their power and decide to put all their money and resources in the direction of their own ideas. Their own ideas as to what a computer should be.

Marketing saw that the business community would be the bigger market. They saw that the typical small businessman went into a computer store, bought an Apple II, a printer, the VisiCalc spreadsheet program, and two plug-in cards. One was a memory card, which allowed them to run larger spreadsheets. And the other was an eighty-column card, which allowed them to present eighty columns of characters across the video display, instead of the normal forty. Forty columns was the limit of American TVs.

So they came up with the idea that this should all be built into a single machine: the Apple III. And it was built.

Initially there was virtually no software designed for the Apple III. Yet there were hundreds of software programs you could buy for the Apple II. So to have a lot of software right away, Apple built the Apple III as a dual computer—there was a switch that let you select whether the computer started up as an Apple II or as an Apple III. (The Apple III hardware was designed to be extremely compatible with the Apple II, which was hard to improve on.) It couldn't be both at once.

And it was here they did something very wrong. They wanted to set the public perception of the Apple III as a business computer and position the Apple II as the so-called home hobby machine. The little brother of the family. But get this. Marketing had us add chips—and therefore expense and complexity—to the Apple III in order to *disable* the extra memory and eighty-column modes if you booted it up as an Apple II.

This is what killed the Apple III's chances from the get-go. Here's why. A businessman buying an Apple II for his work could easily say, "I'll buy an Apple III, and use it in the Apple II mode since I'm used to it, but I'll still have the more modern machine."

But Apple killed the product that businessman would want by disabling the very Apple II features (extra memory and eighty-column mode) he was buying the computer for.

Out of the chute, the Apple III got a lot of publicity, but there was almost nothing you could run on it. As I said, it wasn't reliable. And in Apple II mode, it was crippled.

To this day, it boggles my mind. It's just not the way an engineer—or any rational person, for that matter—would think. It disillusioned me that big companies could work this way.

• o •

Finally, finally, about a year after Apple was able to make the Apple III reliable enough so it wouldn't break constantly, the computer still wouldn't sell. Because by then it had such a bad rep as a terrible, unreliable machine. You see, first impressions matter. When a computer passes by its period of acceptance, you just aren't going to get people jumping on the bandwagon by fixing the problem.

My feeling was, Hey, try to forget about it, and just change the name of the Apple III to the Apple IV and make it look different on the outside, and maybe then you could sell some.

• o •

From the years 1980 to 1983, Apple made the Apple III its highest priority. It's fair to say that Apple became the Apple III company. An Apple III company that just happens to sell Apple IIs.

By 1983 everybody at Apple was forced to have an Apple III on their desk. Suddenly, whenever I walked into the company, they'd be talking like, "Oh my god, did you see such and such new piece of software running on the Apple III?" And it was like, Who cares? I would go around the country in those days and give speeches to computer groups. I would talk to computer groups all over the place, and everywhere I went there would be ninety people with Apple IIs and three people with Apple IIIs.

Why would Apple pretend it was an Apple III company when it wasn't? That was my question.

After all, during these years the Apple II was the largest-selling computer in the world. The Apple II was carrying us. In those days, almost every ad Apple ran in major magazines like *Time* and *Newsweek* showed an Apple III. They never showed an Apple II. The executive staff cut plans for all Apple II products. Totally. There were only a couple of education-related products left.

Despite all this, the Apple II was still paying everyone's salaries and making a huge profit for the company. And it wasn't even being advertised. About the only salary Apple spent on the Apple II during that period—1980 to 1983—was on the guy who printed the price lists.

• o •

It was terrible. I mean, we had everybody at Apple—all the employees and all the money—going into the Apple III and nothing was coming out. And accounting didn't account for it that way. The company lost so much money on the Apple III in those days—in today's money, it would be at least a billion. I calculated at the time that we lost about \$300 million. That's just my own estimate.

And not only was the Apple II carrying the whole company and carrying a debacle like the Apple III, it was hiding the Apple III's real deficiencies from the world. Nobody in the real world, but nobody, treated the Apple III as if it was significant.

All of our users had no idea, I'm telling you. Because if you opened up a computer magazine, all you saw were fifty ads for the Apple II—not by Apple, but by resellers and small mom-and-pop shops who were building all those games and add-ons for the Apple II.

As for the computer magazines, in their reviews of the Apple III, almost every one of them acknowledged it was a failure in the marketplace. Never did they acknowledge that it was a prominent part of Apple's business. They gave consumers the impres-

sion that we were largely an Apple II company—with this hugely successful product—and that there was this big group still working on the flawed Apple III for some reason.

• o •

Now, I accept that Apple had to work the way a company has to. There are a lot of people who operate the company, and there are a lot of people on the board who run things. So the reasoning is very difficult to see. I mean, this was a time when the company had one reputation but it was totally different on the inside. It very much bothered me that you can get away with all kinds of things when you are successful. For example, a bad person can get away with a lot of things if they have a lot of money. And a bad person can hide it—hide behind the money—and keep on being a bad person.

In this case, we had a bad computer, the Apple III, even though the Apple II was selling like hotcakes. It had taken over the world. The IBM PC didn't overtake it until 1983. So it was a leader.

I still don't understand it.

• o •

To be fair, the Apple III had some serious competition. In about 1981, IBM finally came out with its answer to the Apple II. It was selling great almost right away. It was truly becoming a huge success really rapidly. So we had some serious competition all of a sudden, and we'd never had that before.

All those big companies with big IBM mainframe and other large computers were already IBM customers, and it didn't take much for the IBM rep to sell them an IBM PC to go with it all. As a matter of fact, there used to be a saying that "you can't get fired for buying IBM."

When the IBM PC first came out, we were kind of cocky about it. We took out a full-page ad in the *Wall Street Journal* that said, "Welcome IBM. Seriously."

And like I said, the PC passed by the Apple II, the largest-selling computer in the world, in 1983.

• o •

By this time, I should point out, Mike Scott—our president who took us public and the guy who took us through the phenomenally successful IPO—was gone. During the time the Apple III was being developed, he thought we'd grown a bit too large. There were good engineers, sure, but there were also a lot of lousy engineers floating around. That happens in any big company.

It's not necessarily the lousy engineer's fault, by the way. There's always going to be some mismatch between an engineer's interests and the job he's doing.

Anyway, Scotty had told Tom Whitney, our engineering manager, to take a vacation for a week. And meanwhile he did some research. He went around and talked to every engineer in the company and found out who was doing what and who was working and who wasn't doing much of anything.

Then he fired a whole bunch of people. That was called Bloody Monday. Or, at least, that's what it ended up being called in the Apple history books. I thought that, pretty much, he fired all the right ones. The laggards, I mean.

And then Mike Scott himself was fired. The board was just very pissed that he'd done this without a lot of backing and enough due process, the kind of procedure you're supposed to follow at a big company.

Also, Mike Markkula told me Mike Scott had been making a lot of rash decisions and decisions that just weren't right. Mike thought Scotty wasn't really capable of handling the company given the point and size it had gotten to.

I did not like this one bit. I liked Scotty very, very much as a person. I liked his way of thinking. I liked his way of being able to joke and be serious. With Scotty, I didn't see many things fall

through the cracks. And I felt that he respected the good work that I did—the engineering work. He came from engineering.

And as I said, Scotty had been our president, our leader from day one of incorporation until we'd gone public in one of the biggest IPOs in U.S. history. And now, all of a sudden, he was just pushed aside and forgotten.

I think it's sad that none of the books today even seem to recall him. Nobody knows his name. Yet Mike Scott was the president that took us through the earliest days.

• o •

I learned a lot of things at Apple those first few years. I learned right away that in a company, you can have different ideas about what ads look like or what the logo looks like, even different ideas about the name of the company or a product it has. People have different and often conflicting ideas about all these things.

One thing I've learned directly from this new experience of creating and working at a company with so many different people is: Hey, never pretend you can do someone's job better than someone who's been doing it for years.

I was much better keeping quiet and just focusing on my particular talent of engineering. That guaranteed that I would be productive at what I do and could let other people be productive at what they did best.

So few companies were like this. But companies don't always evolve the way you want them to. After all, when we first started Apple, Steve and I really had this engineering-centric model in mind. We wanted Apple to have the amazing employee morale we think HP got as a result of treating its engineers like upper-class citizens, you know?

But we knew what we were getting into because Mike Markkula told us. He said, "This is going to be a marketing company." The product is going to be driven, in other words, by demands that the marketing department finds in customers. This is just the

opposite of a place where engineers just build whatever they love, and marketing comes up with ways to market them. I knew this was going to be a challenge for me.

• o •

Back in high school, I read a book called *The Loneliness of the Long-Distance Runner* by Alan Sillitoe. It just grabbed me. It was about a criminal going through this big mental discussion. It really showed how he was thinking very independently—it showed the way people in general who are inwardly driven think—and he's trying to decide whether he should win this big footrace while he's in jail. The bad governor will become famous if the criminal wins.

And he's trying to decide, Should he win the race, or shouldn't he? Should he let the governor have all the fame? Or should he try to run away, and keep running, and just escape?

The whole thing had a huge impact on my own thinking. In life, there is an "us" and a "them." A "we" and a "they." And the "they" is the administration, the authorities. And sometimes they're on the wrong side and we're on the right side.

Chapter 16

Crash Landing

Before Alice and I were divorced, Alice told me about a friend of hers, Sherry, who was interested in buying a movie theater. A real running theater. It was the Mayfair Theater in San Jose. Alice thought I should buy it, and I could never turn Alice down when it came to anything she wanted to do.

So I bought it.

Sherry and Alice had gotten involved with a group called Eastern Star, a group of women who had relatives in the Freemasons. Because she was in Eastern Star, she was spending a lot of time there, a lot of nights away there. In order to have more time with her, I decided that I would become a Freemason. Freemasons, after all, regularly have joint events with Eastern Star. So I went down to the Masonic lodge and did a lot of training, and after some period of time and three big events, I became a third-degree Mason. Then I got more time with Alice. I eventually became an officer and everything.

I should tell you that although I am a lifetime Freemason, I'm not like the other people who are Freemasons. My personality is very, very unlike theirs. To get in, you have to say all this stuff about God, the Bible, words that sound a little bit like they come from the Constitution, and none of this ritual stuff is the way I

think, you know? But I did it, and I did it well. If I'm going to do something, I always try to do it well. And I did this for one reason, as I said: to see Alice more. I wanted to save the marriage. I would go so far as to join the Freemasons if that's what it took. That's how I was.

So anyway, pretty much near the end of the marriage, I was a Freemason and I bought that theater. Alice's friend Sherry and Sherry's boyfriend, Howard, would run it. It had been their idea from the start, to run a theater. They'd gotten to Alice, as a friend, so she got to me. And now I owned it.

The Mayfair Theater was in kind of a low-income area of town. I remember we had to paint the bathroom black because of all the graffiti, and even afterward, people would still put graffiti in it, only in white paint. At least we could wash the walls.

I felt like making it into something special. I never had the idea that it was going to make a lot of money, but I wanted it to be kind of special and I put in nice seats and a good sound system. I had a couple of guys running it, and they scraped off a wall one day and found there was this beautiful natural wood artwork underneath this blah wall someone had tacked up on top of it. So we actually brought in some experts who sanded everything down, and they were able to recover the original artwork. I loved that theater.

But then Alice and I got divorced, and I was stuck with the theater. I went there every day after work at Apple. I drove down there, set up my computer so I could get some work done, saw what movies were playing, and said hi to everybody. The theater was this fun group of people, a really small operation. It was neat to see how it operated. I mean, it was a small, low-budget theater. We didn't get that many customers. And we only got pretty low-rate movies. For instance, we had *Friday the 13th*. That was probably the biggest movie we ever showed, and we only got it long after it opened.

Actually, the only movies we ever sold out on were gang movies, like *The Warriors*. That made sense, considering what part of town we were in!

I'd only been single a few weeks when I asked out the woman who would be my second wife, Candi Clark. I knew her because once, when I bought a bunch of advance tickets to a *Star Trek* movie and offered them half price to Apple employees, she'd asked for a bunch because she had a lot of brothers. I thought she was pretty cute, so I asked her to come to one of those low-budget science fiction movies we were showing at my theater, and she did. The next day, we raced bumper cars at the Malibu Grand Prix track near the San Francisco airport and I beat her really well.

I thought she was just super pretty. She was blonde, medium build, and it turned out she had been an Olympic kayaker. (I found that out when I saw a picture of her and Ronald Reagan on the wall of her apartment after our second date.) She worked at Apple creating database reports for managers, that sort of stuff.

So now I had a girlfriend and that was it. It was all really quick.

• o •

It wasn't very long after I'd divorced Alice and met Candi that we decided to get married. She had an uncle down in San Diego who made jewelry, and I had this idea. Let's get a ring for me, I said, that has the diamond on the inside so nobody can see it. I thought that would be more special than a normal ring. We would know there was a diamond, but the world wouldn't.

So we decided to fly down on a plane, on my V-tail Beechcraft, which I'd bought right after getting my pilot's license six months before. I think today that it was the most beautiful and unorthodox single-engine plane there is. It was so distinctive, the shape of its tail was so unique, and I was so proud to fly it. I had it painted—by a painter named Bill Kelly, he'd done PR for Apple—in the nicest earth tones.

The first time in my life that I was able to take a passenger

alone, it was with Candi. I took her down to San Jose one night and it was raining. Of course, I had never flown in the rain at night, but I did and we got back safely. I think that might have been my best landing ever.

But no, I wasn't at all cocky about my flying. I knew how to do a flight plan and how to do flights. I knew the rules to follow. But still, I was a beginner pilot. I was still a pretty rough new trainee. But anyway, Candi and I took a few trips in the new plane, and then one day we decided to fly down to San Diego where Candi's uncle could design that wedding ring with the diamond on the inside.

Candi and I flew from San Jose to a small airport in Scotts Valley to pick up Candi's brother Jack and Jack's girlfriend, Chris. Usually I would just taxi around and then take off, you know? So I'm going around, and suddenly I notice I'm blocked by another plane that's just sitting there, stalled on the taxiway. I'm thinking, Great. Great. I can't even get out of there.

So I looked around—I think we turned the airplane around—and I go off some other side way. By then the stalled airplane was gone and finally I got to the start of the runway. And I did all the little start-up procedures and reached for the throttle and you know what?

I remember reaching for the throttle at the start of the runway, and that's it. I can remember every other detail of the airport and everything that day up to that point. But I can remember absolutely nothing about what happened after that point. I have no memory of what happened next. (Later, I figured out that maybe Candi, who was sitting in the front, accidentally leaned on one of the controls, but we'll never know exactly what caused that accident.)

I woke up in the hospital, so they tell me, but it wasn't until five weeks later that I was able to remember that I was in a plane crash.

My friend Dan Sokol later told me that he saw news of the accident on TV. He said he turned on the TV and clicked onto the news channel when he heard something about an executive of a Silicon Valley computer company crashing his plane in Scotts Valley. And he immediately turned around just in time to see about two seconds of the Beechcraft upside down. I had crashed in the parking lot of a skating rink.

Of course, as I told you, I remember absolutely nothing about what happened, not even about being in the hospital or anything. It was some head injury! Dan told me my room was filled with gifts and toys and stuff from people at Apple. Handmade cards, off-the-shelf cards, and junk food. It was all there, Dan said, but I have no memory of it. Zero memory. Dan even told me that I asked him to smuggle in a milk shake and pizza for me, which sounds exactly like me, so at least I know that I was really in there. I mean, people took pictures of me in there playing computer games, which is what I would do, but I have no memory of that. No memory at all.

At some point, I guess a week or two later, I was finally released and allowed to go home. I didn't go to Apple to work, I presume because I thought every day was a weekend. That's the only explanation I can think of now as to why I didn't go to work, and also why I didn't notice my dog was missing. (He'd been checked into a kennel.)

For a few weeks after, I was living in my house in Scotts Valley in this weird, not-fully-functional state. I mean, people later told me I seemed hazy. They say I was driving around on my motorcycle, but people really had to direct me to do things. Like: "You go here. You have to do this now. Now you have to do this." I was apparently functioning, but I hardly have any memories of it. I was living this halfway weird life. I didn't realize that my dog had been boarded for five weeks away from me, for instance. It just seemed like every day was the same day. I didn't

even realize I was missing a tooth for five weeks—one of my front teeth! How do you not spot something like that? I don't know, I can't explain it.

Now, Candi and her brother, I found out much later, were also injured in the crash. She even had to get some plastic surgery afterward. But I was the one who was the hardest hit. As I said, I ended up having what is known as anterograde amnesia, even though the doctors didn't know it at first. Anterograde amnesia means that you don't lose memories; you just lose the ability to form new ones.

But I guess, when I think about it now, it was actually a good thing because in my mind, I never had a plane crash to get over. It just isn't there. I underwent hypnosis to see if I could come up with any recollection of what happened to cause the crash. I really would've liked to know. But nothing came to me.

So in those five weeks—the weeks of my amnesia—I remembered everything from before that. I had all my old skills and memories, and those memories are still there up till that point. But during that five-week period, whatever I was doing, I wasn't remembering it.

And then suddenly I came out of it.

The first, the very first, memory I had was that I was somehow at the Macintosh building talking to associates I'd been working with on the Macintosh. And they were telling me something about how the project was going. And I don't remember exactly who, but I think it was Andy Hertzfeld (designer of the Macintosh graphical user interface) who mentioned something about a plane crash. A plane crash? And the instant he said the words "plane crash," I knew there was this thing about a plane crash in this dream I'd been having.

So I said to myself, Oh, this is a dream I'm having right now. And in a dream, I can always tell myself that I can just turn around and walk the other way. You can go any which way and a

dream follows you. But this time I thought, No, I'll play by the rules of this dream and I'll keep talking to Andy. So I sat there talking to him, and that's my very first memory. But it was a very weak memory.

That night, I remember Candi and I went to see the movie *Ordinary People*. I don't remember a single detail of that movie, only that we saw it. Then we got home and we were in bed. I was lying on my back and thinking, Wait, did I have a plane crash that I heard about and kept dreaming about, or didn't I? I mean, I didn't have any memories of such a crash, and it seems like you would remember such a thing, wouldn't you?

Is it possible I had a plane crash and didn't remember it?

So I turned over and asked Candi, "Did I have a plane crash or was it a dream?"

I guess she thought I was joking, because she said, "It was a dream, Steve." That's what she said. That it was a dream. She wasn't playing with my head. She just had no idea that I had no idea I'd been in a plane crash.

This was a mental dilemma because I was struggling to prove in my head that it could be true.

So now I'm sitting there wondering if I'm ever going to get anybody to tell me if I had a plane crash or not. I suppose if I'd been smart, I would have looked in the newspaper or asked other people, but this was actually the first time I was starting to think that maybe I had in fact had a plane crash and it wasn't a dream.

So I sat there that night, feeling my body. And my body didn't have any broken bones or signs of a plane crash. Ha. I didn't think to look for a missing tooth!

So I kept thinking. I kept trying to pin it down. How do you figure out if something didn't happen? I could remember every single detail of that day up to the point of reaching for the throttle, but I couldn't remember pushing it. And then I thought of something logical. I thought, Wait a minute. I don't remember landing